Contents

Introduction....4
What is a Playwright? 4
Used Technologies – NodeJS, TypeScript, Playwright, VSCode....5
What is a programming language?....5
What are syntaxis and semantics in programming languages? 5
What is JavaScript (Vanilla JS)?....6
What is a JS Engine & NodeJS? 7
What is Framework?....7
What is a testing framework, and why do we need it (What is a testing framework)? 8
A testing framework is a set of tools and libraries that help us write and execute tests automatically. It provides structure and a standard way to create, run, and report tests.....8
Why do we need a testing framework?....8
1. Organization: The framework offers a structure for organizing tests logically (e.g., test files, classes, and methods). 8
2. Automation: It provides tools for automatically running tests across multiple environments, browsers, and platforms. 8
3. Reporting: It generates automatic reports on successful and failed tests.....8
4. Easier maintenance: The framework simplifies updating and adding new tests. 8
Example of a testing framework: 8
Playwright is one of the popular testing frameworks for browser automation.....8
What is IDE?....8
What is a Boilerplate?....9
README.md file.....9
package.json....10
Dependency (external resource) and Scripts....10
paclage-lock.json 11
tsconfig.json 11
What is TypeScript? 12
Main playwright file – playwright.config.ts....13
Eslint, Prettier....13
OOP Stuff....14
OOP Principles 14
What is Variable, What is Variable Type, What is a function, What is a class?....17
Asynchronous and Non Asynchronous data....18
Domain Specific Language....20

Short story with a simple example 20
What exactly are we doing in common.ts? 21
Why do we need to group Playwright functions? 21
Page Object Model Design Pattern 22
What is Design Pattern, and why is it good to be used in programming? 22
Why is the POM Design Pattern's most helpful design pattern in Automation Testing? 23
How is it implemented in our testing framework? 24
What is the idea of a base Page? Why are we using Base Page? 24
What is an element? 25
What is an attribute? 26
What is an attribute value? 26
What is a DOM tree? 26
What is a locator? 27
Different ways to locate an element. The best and worst approaches for locating an element (from ID to Class) 27
Stable locators, and why do we need a stable locator? 28
A real example of creating a locator for elements used in the example. 30
Creating POM function. 30
Showing a real example of a function. 30
Test Page 32
What is the idea of a test page? 32
Show an example of the structure of the test class. 33
The main block of the testing framework. The priority of executing different test blocks 33
Testing Data. Different approaches to testing data. 33
Static testing Data. 33
Dynamic testing data 34
Calling the functions defined in the POM class. 35
Debugging 35
What is debugging 35
Why is debugging in JS so hard (because compilation happens before the execution)? 36
What is our solution for debugging? 37
Executing the tests from the terminal. 37
Creating new script records inside the package.json. 38
Automatically generated Reports after executing the tests. 39
Grouping automation tests in Suites 39
Execute different suites. 39

API testing using Playwright. 39
Combining API tests and Web Automation tests. 40
Updating the packages in the NodeJS project. 42

Introduction

This course can serve as a good foundation for getting familiar with Playwright.
The course includes both a lecture section and a practical section.
The practical part contains references to a prepared boilerplate, which can be used as a great starting point for any new project. The boilerplate itself is rich in comments and documentation, making it easier for students to understand its structure.
The boilerplate can be found here: https://gitlab.diva-e.com/quality-agents/playwright-boilerplate

The lecture section mainly includes lessons on JavaScript, TypeScript, NodeJS, and Playwright. Some of the topics are for complete beginners, while others are for more advanced learners.

As mentioned earlier, the course can serve as a solid foundation that students can use to begin automating with Playwright and TypeScript. The course covers all the best practices for automated testing.

What is a Playwright?

Lecture Level: Junior

What is a Playwright?
Playwright is a powerful browser automation tool developed by Microsoft. It allows developers and QA professionals to automatically test web applications across browsers, including Chromium, Firefox, and WebKit. With Playwright, you can write tests in JavaScript, TypeScript, Python, C, and Java. Its main goal is to simplify the testing of web applications by providing powerful tools for automation and functionality verification.

Key Features of Playwright:
1. Multi-browser support:
Playwright supports three major browsers:
- Chromium (the core engine for Chrome and Edge)
- Firefox
- WebKit (the core engine for Safari)

This allows you to test the same application with minimal code changes across different environments.

2. Cross-platform support:
Playwright can run on various operating systems, such as Windows, macOS, Linux, Android, and iOS.

3. Asynchronous tests:
Playwright supports asynchronous tests, making it well-suited for dynamic web applications where delays in loading content or making API calls may occur.

4. Test environment isolation:
Playwright allows you to isolate your tests by creating a new browser context or instance for each test. This allows tests to run independently of each other, preventing potential conflicts and making them more reliable.

5. Screenshot and video recording:

Playwright allows users to automatically take screenshots or record videos during tests, which helps diagnose errors.

6. Multiple tabs and frames:
Playwright easily handles multiple browser tabs or iframes in a single test, providing APIs for interacting with them.

7. Powerful selectors:
Playwright offers various methods for locating elements on web pages, supporting CSS selectors, XPath, and even Playwright-specific locators like `text=`, `role=`, and others.

8. Combined execution of API and Web tests:
Playwright allows you to combine API tests with web tests in the same test scenario. This means you can send HTTP requests to test the backend logic of the application (API) and automate interactions with web pages in the browser.

How does combined execution work?
- API Test: The test starts by sending a request to the server (e.g., POST, GET, PUT, DELETE). You can verify the server response by analyzing the response data (e.g., whether the request was successful or the data is correct).
- Continue with Web Test: After sending the API request, the browser simulates interaction with the web page. For example, after sending an API request, you can verify that the web page correctly displays the information returned from the server.

Combining API tests with web tests is helpful because it allows you to skip specific scenarios tested separately in other tests. For example, if most tests require login, running this test through the web might take longer than executing it via an API request. Of course, login should be tested at least once through the web, but for other scenarios where login isn't the focus, you can use an API request to perform it using a method called Silent Login (Authentication).

How is Playwright used in testing?
Playwright automates functional tests to verify that a web application works correctly across all supported browsers. It is beneficial for end-to-end testing, where you check how users interact with the application.

Used Technologies – NodeJS, TypeScript, Playwright, VSCode

What is a programming language?

Lecture Level: Junior

A programming language is a means of communication between humans and computers. It allows developers to write instructions (code) that the computer can understand and execute. Programming languages use specific rules (syntax and semantics) to tell the computer what to do – such as displaying text, calculating numbers, or sending data.

What are syntaxis and semantics in programming languages?

Lecture Level: Junior

Syntax and Semantics with TypeScript:
1. Syntax:
Syntax refers to the rules for writing code—how it must be structured so the computer can understand it.

Example with TypeScript:

```
let name = "John"
```

If the variable is not defined in this way, you will get a syntax error.

2. Semantics:
Semantics describes the meaning of the code – what the program actually does when it runs. While syntax tells us how to write the code, semantics shows what actions are performed.

Example with TypeScript:

```
let a = 5
let b = 10
let sum = a + b
```

The semantics here are that the numbers `a` and `b` are added together, and the result is stored in the variable `sum`. The meaning of this code is that it calculates the sum of two numbers.

Difference between Syntax and Semantics:
- Syntax: How the code should be written (rules).
- Semantics: What the code means and what it does (logic).

If the syntax is incorrect, the code won't run. If the semantics are incorrect, the code might run but return the wrong result.

What is JavaScript (Vanilla JS)?

Lecture Level: Junior

JavaScript (JS)
Before we explore any technology using NodeJS, we need to clarify the foundation, which starts with JavaScript.

JavaScript is one of web development's most popular and widely used programming languages. It is a scripting language that allows developers to add dynamic behavior to web pages, making websites interactive. Unlike HTML and CSS, which describe the structure and styling of a web page, JavaScript enables control over the logic and behavior of the pages.

Key Features of JavaScript:
- Client-side language: JavaScript is typically executed on the client side (in the browser) and is used for user interaction, DOM (Document Object Model) manipulation, form validation, and other actions. We will learn more about DOM in future lessons.
- Asynchronous: JavaScript supports asynchronous operations, meaning it can perform tasks like making server requests without blocking the rest of the program. In upcoming lessons, we will cover asynchronous behavior more.
- Flexible and dynamic language: JavaScript is dynamically typed, meaning variables do not require type declarations at compile time. Here are a few examples of declaring dynamically typed variables:
JavaScript:

```
const variableString = `this is a test`;
let variableNumber = 3;
var variableBoolean = true;
```

In these examples, JavaScript automatically assigns the data type based on the value provided. We will explore the language in more detail later.
- Cross-platform: JavaScript can run on various platforms, including mobile devices, desktop applications, and servers (with the help of Node.js).

Main Areas of Application:
- Web development: Adding interactivity to web pages, such as form validation, dynamically loading content, animations, and more.
- Server-side development: Platforms like Node.js and JavaScript can also be used for server-side programming, providing backend functionality.

Vanilla JavaScript:
Vanilla JavaScript uses pure, unextended JavaScript without additional libraries or frameworks like React, Angular, or jQuery. It is the core, "natural" JavaScript as defined by the ECMAScript standard.

What is a JS Engine & NodeJS?

Lecture Level: Junior

What is a JavaScript Engine?
A JavaScript Engine is a software execution environment that interprets and runs JavaScript code. It is the core of any application that uses JavaScript and is responsible for translating high-level JavaScript code into machine code that the computer can execute.

Node.js and the JavaScript Engine (V8)
Node.js is a runtime environment for JavaScript that operates outside the browser. The critical difference between using JavaScript in a browser and Node.js is that Node.js uses the V8 JavaScript Engine, which can be extended with libraries to handle file systems, network requests, and other server-side functionalities.

What is Framework?

Lecture Level: Junior

What is a Framework?
A Framework is a pre-built software structure or platform that provides a set of libraries, tools, and development guidelines. It is a foundation for developers to build software solutions without starting from scratch.

Key Features of a Framework:
1. Defines the application's structure: A framework provides guidelines and structures to organize the code, helping developers follow best practices and avoid mistakes.
2. Code reuse: A framework offers ready-made components and functionalities, reducing the need to write repetitive code.
3. Integration with various technologies: Many frameworks make it easier to integrate different technologies (databases, web servers, APIs, etc.).
4. Easy maintenance and scalability: Applications built with frameworks are easier to maintain and scale over time due to their clear structure and modularity.

Examples of Frameworks:
- Frontend Frameworks: These are used to build the client side of web applications.
  - React: A popular JavaScript framework for creating interactive user interfaces.
  - Angular: A platform for building dynamic web applications.
  - Vue.js: A lightweight and flexible JavaScript framework for building user interfaces.
- Backend Frameworks: These are used to build the server side of applications.
  - Express.js: A popular Node.js framework for building web servers and APIs.
  - NestJS: A framework for NodeJS that facilitates rapid API development.

Why do we use Frameworks?
- Speeds up development: Frameworks offer ready-made solutions to common problems in software development, saving time and reducing errors.
- Security: Many frameworks provide built-in security mechanisms (e.g., protection against SQL injections, session security, etc.).
- Standardization: Frameworks encourage using widely accepted standards and best practices, making applications easier to maintain and develop.
- Community support: Most frameworks have large, active communities, which make it easier to find examples, tutorials, and solutions to problems.

How is a Framework different from a Library?
A library is a collection of functions and methods that can be used within an application. A framework, however, is more comprehensive—it provides functions and defines the structure and how the code should be written. In this sense, the framework "controls" the application flow, while the developer has complete control over the code with a library.

Example:
- If you use a library like jQuery, you decide when and how to use its methods.
- With Angular (a framework), your application follows a specific model and structure, with the framework managing how and when the code is executed.

What is a testing framework, and why do we need it (What is a testing framework)?

Lecture Level: Junior

A testing framework is a set of tools and libraries that help us write and execute tests automatically. It provides structure and a standard way to create, run, and report tests.

Why do we need a testing framework?

1. Organization: The framework offers a structure for organizing tests logically (e.g., test files, classes, and methods).
2. Automation: It provides tools for automatically running tests across multiple environments, browsers, and platforms.
3. Reporting: It generates automatic reports on successful and failed tests.
4. Easier maintenance: The framework simplifies updating and adding new tests.

Example of a testing framework:

Playwright is one of the popular testing frameworks for browser automation.

What is IDE?

Lecture Level: Junior

What is an IDE?
An IDE (Integrated Development Environment) is a software application that consolidates all the tools necessary for writing, testing, and debugging code in one environment. It helps developers create software more easily and quickly.

Key features of an IDE:
1. Code editor: The place where you write your code. It highlights syntax and provides auto-completion suggestions.
2. Compiler or interpreter: Translates the code into a language the computer understands so the program can run.
3. Debugger: A tool that helps you find and fix errors in the code.

4. Version control integration: This makes it easier to manage different versions of a project (e.g., using Git).

Examples of IDEs:
- Visual Studio Code (VSCode): Lightweight and easy to use, viral for web development.

Conclusion:
An IDE is an indispensable tool for programmers. It combines everything needed for development in one program, simplifying the entire process of writing and testing code.

What is a Boilerplate?

Lecture Level: Junior

What is a Boilerplate?
The term "Boilerplate" in programming and software development refers to pre-written code that can be reused without changes or with minimal adaptations. This code is often used as a starting point for new projects or as a foundation for developers to add project-specific features.

Why Use Boilerplate Code?
1. Efficiency: Using boilerplate code saves time and effort, as developers do not need to rewrite commonly used functional code.
2. Reduced Error Likelihood: Since boilerplate code is usually well-tested and widely used, its likelihood of containing errors is smaller than newly written code.
3. Consistency: It provides a uniform approach to developing different parts of a project or across various projects, which improves the code's readability and maintainability.

How to Use Boilerplate Code?
Boilerplate code can be extremely useful, especially when starting a new project or working with a new technology. The best way to use it is to incorporate it into your project and then adapt it according to its specific requirements. It is important to understand how the boilerplate code works so that you can effectively modify it and solve problems that may arise.

Using boilerplate code saves time when starting projects, promotes good coding practices, and facilitates collaboration among developers as the code becomes more predictable and easier to understand.

!!! Important !!!
A boilerplate for Playwright is provided, which is located in the company's GitLab repository. Some of the lessons here are directly related to this boilerplate and serve as practical examples. The boilerplate can be used for new projects and can be a good starting point for a new project.

README.md file.

Lecture Level: Junior

Since we know that we have a sample boilerplate to examine, we have a project with which we are unfamiliar. This means that we need to familiarize ourselves with it. Everything necessary for the project should be described in the README.md file.

But let's understand a little more about README.md files in general.

What is a README.md File?
A README.md file is a document that typically accompanies software and code repositories, especially in platforms like GitHub, GitLab, or Bitbucket. The "md" extension stands for Markdown, a lightweight markup language used to format the text in the file. This file is crucial for software development and code sharing because it offers a first explanation or overview of the project. It's essentially the first documentation someone sees when they encounter your code.

Purpose of a README.md File
The primary purposes of a README.md file include:
1. Introduction and Overview: It introduces the project to potential users or new developers and provides a high-level description of what the project does and its purpose.
2. Instructions: It commonly includes instructions on installing, configuring, and using the software. This might consist of steps to compile the code, how to set up the environment, and how to run the application.
3. Documentation: The README.md often links to more detailed documentation or wikis if the project is complex. It can also include examples of how to use the software, code examples, and other important information.
4. Contact Information: It might provide contact information for the creator or team behind the project or directions on how to contribute to the project.
5. License Information: It usually includes a section detailing the project's license, which informs users about how they can legally use, modify, and share the project or software.

Why is it a README.md Important?
A well-crafted README.md file is crucial because:
- It's often the first thing users see in a repository, creating a significant first impression.
- It guides new users in using, installing, or contributing to the project, making the software more accessible to a broader audience.
- It improves the professionalism of the project, demonstrating care and effort in development.

Markdown (used in README.md files) supports various formatting options such as headers, lists, links, and code blocks, making it versatile enough to create comprehensive and well-organized documentation.

You can view the README.md file of the boilerplate project for more details.

package.json

Lecture Level: Junior

You can view the package.json file of the boilerplate project.

Dependency (external resource) and Scripts

Lecture Level: Junior
We are now starting with NodeJS. As we already learned, NodeJS is a JavaScript engine that allows JavaScript code to run without a browser. JavaScript was initially designed to run in a browser environment, but it is a versatile programming language that can be used for other purposes. Therefore, we need an engine that allows JavaScript code to execute outside of a browser.

In NodeJS, one central file, called `package.json`, is the project's leading file. It contains information about all the dependencies that the project needs to function. These dependencies are in the `dependencies` and `devDependencies` sections. Dependencies are pre-written code that provides specific functionalities that are ready to be used. Using dependencies is a best practice because it simplifies software development, reduces development time, and helps with maintenance. This is because using a library (or "package" in NodeJS) allows you to use ready-made functions or features, with the maintenance of that package being handled by its creators.

Another important section in the `package.json` file is `scripts`. In this section, we can define different terminal commands that can be executed with a short keyword that we specify. We do this because sometimes the command to start an application (or tests) can be extended. NodeJS allows us to define and execute long commands with a shorter, custom-defined keyword.

For example, if the command to run a test for registering a new user is:

```
playwright test RegisterNewUser.test.ts
```

You must type this long command every time to run the test. Instead, you can define a new NodeJS script command in the `scripts` section of the `package.json` file. It might look like this:
JSON syntaxis:

```
"scripts": {
  "test:register": "playwright test RegisterNewUser.test.ts"
}
```

This way, in the terminal, you can simply run:

```
npm run test:register
```

paclage-lock.json

Lecture Level: Junior

What is package-lock.json?

package-lock.json is a file automatically created when you install dependencies (libraries) in your Node.js project using `npm install`. It records the exact versions of all installed packages and their dependencies.

What is this file used for?

1. Ensures stability: It locks all library versions to ensure that everyone working on the project will use the identical versions. This prevents issues that might arise if different versions of a library behave differently.
2. Faster installation: During future installations, npm uses the information in package-lock.json to load packages faster instead of searching for new versions online.
3. Security: The file also shows all the dependencies the libraries use and their versions, which is essential for avoiding vulnerabilities in the code.

Conclusion:

package-lock.json provides stability and predictability for your project by ensuring that all libraries will be installed with the exact versions the project was developed with. This is crucial for team collaboration and avoiding errors in different package versions.

You can view the package-lock.json file of the boilerplate project.

tsconfig.json

Lecture Level: Junior

tsconfig.json is a configuration file used in projects written with TypeScript. It instructs the TypeScript compiler on how to behave when converting TypeScript code into JavaScript.

What is tsconfig.json used for?

1. Configuring the compilation: This specifies precisely how the TypeScript code should be compiled. You can set various rules and settings for the compiler, such as:
- The version of JavaScript to compile to (e.g., ES5 or ES6).
- Where to output the compiled JavaScript files.
- Whether to enforce stricter type-checking rules.
2. Selecting files for compilation: You can define which files or folders should be compiled with this file. For example, you can exclude specific files or directories if you don't want them included in the compilation.

Key settings in tsconfig.json:

Here is an example of a basic tsconfig.json file:

JSON:

```
{
  "compilerOptions": {
    "target": "ES6", // Compiles TypeScript code to ES6 version of JavaScript
    "outDir": "./dist", // The folder where the compiled files will be saved
    "strict": true, // Enables stricter type-checking
    "module": "commonjs" // The module system to be used (for Node.js)
  },
  "include": ["src//*"] // All files in the src folder will be compiled
}
```

Why is tsconfig.json important?
- Customization of compilation: This file allows you to fine-tune how the compiler behaves, such as using a specific JavaScript version or including only particular files.
- Maintaining project structure: In larger projects, tsconfig.json helps organize and control how and which files are compiled, keeping the project well-structured and manageable.

You can view the tsconfig.json file of the boilerplate project.

What is TypeScript?

Lecture Level: Junior

What is TypeScript?
TypeScript is a programming language that extends JavaScript. It adds static types to JavaScript, helping developers catch errors while writing code rather than waiting until runtime.

How does TypeScript work?
- Data Types: In TypeScript, you can specify the types of variables, functions, and objects, such as whether a variable is a number, string, or array. This makes the code more stable and helps prevent errors.
- Compilation: Browsers cannot execute TypeScript code directly. Therefore, it is compiled (translated) into JavaScript and then executed. This means that you can do anything with JavaScript and TypeScript.

Why use TypeScript?
1. Fewer Errors: With the help of types, TypeScript helps catch errors during coding before you even run the application.
2. More Readable and Organized Code: Using types makes the code easier to understand and maintain, especially in large projects.
3. Works with JavaScript: All JavaScript programs are valid TypeScript programs, meaning you can start with JavaScript and gradually add TypeScript to your project.

Example with JavaScript:

```
function sum(a, b) {
  return a + b;
}
```

Example with TypeScript:

```
function sum(a: number, b: number): number {
  return a + b;
}
```

If you try to execute `sum("1", 2)` in TypeScript, you'll get an error because the first argument should be a number, not a string. This way, data type errors are caught during coding, not at a later stage of software development.

Main playwright file – playwright.config.ts

Lecture Level: Junior

What is playwright.config.ts?
playwright.config.ts is a configuration file used in Playwright that contains settings for how Playwright will run automated tests. This file allows you to configure various aspects of testing, such as browsers, timeouts, file paths, and more.

What is playwright.config.ts used for?
1. Setting browser options: In this file, you can configure which browsers to use for tests (e.g., Chrome, Firefox, Safari). The Playwright supports testing across multiple browsers simultaneously.
2. Timeout settings: You can define how long Playwright should wait for a test or action to complete before marking it as failed. This helps manage cases where certain operations may take longer than expected.
3. Parallel test execution: In playwright.config.ts, you can specify how many tests should be run simultaneously (in parallel), which speeds up the overall testing process.
4. Test file paths: You can specify which directories or files Playwright should look for and run tests. This helps you organize your project more effectively.
5. Reports and screenshots: You can configure Playwright to generate test reports or take screenshots when errors occur, making analyzing what went wrong during a failed test easier.

You can create multiple configuration files (with different names), but each should include .config.ts in its name. The configuration file is read before the tests are executed, allowing Playwright to understand how to run the tests properly.

You can view the playwright.config.json file of the boilerplate project.

Eslint, Prettier

Lecture Level: Mid

What is ESLint?
ESLint is a tool that analyzes your JavaScript or TypeScript code and identifies potential errors or bad practices. By highlighting issues with syntax or coding style, it helps you keep your code clean and consistent.

What does ESLint do?
- Finds errors: It checks your code for syntax and logical errors, pointing them out as you write.
- Enforces style rules: ESLint helps you follow a consistent coding style, alerting you to inconsistent variable names, improper formatting, or other harmful practices.
- Customizable rules: You can configure ESLint with a set of rules you or your team prefers to ensure consistency across the codebase.

Example with JavaScript:
If you forget to define a variable in your code, ESLint will show you a warning or an error to fix it:

```
let x = 10;
console.log(y); // ESLint will flag this as an error because 'y' is not defined
```

What is Prettier?
Prettier is a tool that automatically formats code. It ensures that your code is neat and easy to read by applying formatting rules, such as indentation, line breaks, alignment, and more.

What does Prettier do?
- Automatically formats code: Prettier doesn't look for errors in the code but simply organizes it according to predefined formatting rules. When you save your file, Prettier can automatically adjust the formatting so that all lines are aligned and consistent.
- Maintains consistency: No matter who works on the project, Prettier ensures that everyone follows the same format, making the code easier to maintain and read.

Example with JavaScript:
The following code is manually written without following formatting rules:

```
let x=10; function add(a,b){return a+b;}
```

After formatting with Prettier, it looks like this:
Example with JavaScript:

```
let x = 10;
function add(a, b) {
  return a + b;
}
```

Differences between ESLint and Prettier:
- ESLint: Finds logic and syntax errors and violations of style rules.
- Prettier: Only formats the code to make it look clean and readable.

These two tools are often used in a project—ESLint for finding errors and enforcing style and Prettier for automatic formatting.

You can view the .prettierrc file of the boilerplate project. This is the Prettier configuration file. And you can view the .eslint.config.mjs file of the boilerplate project. This is the ESlint configuration file.

OOP Stuff

OOP Principles

Lecture Level: Junior

Key Principles of OOP
Object-oriented programming (OOP) is a programming paradigm in which software is modelled around objects. Each object represents something specific and has properties (attributes) and actions (methods).

1. Classes and Objects:
- Class: This is a blueprint or "template" for an object. Think of it like a house plan—it contains all the characteristics but is not the house itself.
- Object: This is the actual house built from the plan. In other words, an object is an instance of a class.

The analogy with the Human Body: We can have a class "Person" that defines that every person has legs and can walk. The object would be a real person, like you or me, who uses their legs to act for walking.

Example with TypeScript:

```
class Person {
  // class properties
  legs: number;

  constructor() {
    this.legs = 2; // Every person has 2 legs
  }

  // class method
  walk() {
    console.log("The person is walking with their legs.");
  }
}

// Create an object (real person) from the Person class
const john = new Person();
john.walk(); // "The person is walking with their legs."
```

2. Inheritance:
Inheritance allows one class to inherit properties and methods from another class, saving time and avoiding duplication of code.

The analogy with the Human Body: Imagine we have a class "Person," but we want to create a class "Athlete" that inherits all the characteristics of "Person" (legs, arms, etc.) but also has additional skills like "running."

Example with TypeScript:

```
// Base class "Person"
class Person {
  legs: number;

  constructor() {
    this.legs = 2;
  }

  walk() {
    console.log("The person walks.");
  }
}

// "Athlete" class inheriting from "Person"
class Athlete extends Person {
  run() {
    console.log("The athlete runs fast.");
  }
}

// Create an object from the "Athlete" class
const athlete = new Athlete();
```

```
athlete.walk(); // "The person walks." (inherited from Person)
athlete.run();  // "The athlete runs fast." (new action)
```

3. Encapsulation:
Encapsulation means that the object's data is hidden and can only be accessed through methods that provide data protection.

The analogy with the Human Body: Think of the human heart as encapsulated – it is hidden inside the body, and you cannot directly control it, but you can influence it through actions like running (which increases the heart rate).

Example with TypeScript:

```
class Person {
  private heartRate: number; // encapsulated property

  constructor() {
    this.heartRate = 70; // normal heart rate
  }

  run() {
    this.heartRate += 30;
    console.log("The heart beats faster: " + this.heartRate);
  }

  getHeartRate() {
    return this.heartRate; // method to access encapsulated data
  }
}

const person = new Person();
person.run(); // "The heart beats faster: 100"
console.log(person.getHeartRate()); // Access the encapsulated data: 100
```

4. Polymorphism:
Polymorphism allows different classes to use the same function in various ways.

The analogy with the Human Body: If you have two different types of people – one who strolls and another who walks quickly – they are using "walking" differently.

```
Example with TypeScript:
class Person {
  walk() {
    console.log("The person walks.");
  }
}

class Athlete extends Person {
  walk() {
    console.log("The athlete walks very fast.");
  }
}
```

```
const regularPerson = new Person();
const athlete = new Athlete();

regularPerson.walk(); // "The person walks."
athlete.walk();      // "The athlete walks very fast." (different implementation)
```

What is Variable, What is Variable Type, What is a function, What is a class?

Lecture Level: Junior

What is a Variable?
A variable is a location where information (data) is stored in a computer's memory. You can think of a variable as a "box" where you can place a value you will use later in the program.

Example with JavaScript:

```
let age = 25; // We create a variable "age" that stores the value 25.
```

What is a Variable Type?
The type of a variable determines what kind of data the variable holds. In programming, there are different data types, such as:
- Numbers (number): Whole numbers or decimals.
- Text (string): Textual values (words, sentences).
- Booleans (boolean): Values of "true" or "false".

Examples of different types:
Example with TypeScript:

```
let age: number = 25; // A number variable
let name: string = "John"; // A string variable
let isStudent: boolean = true; // A boolean variable
```

What is a Function?
A function is a block of code that performs a specific task. You can think of a function as a "recipe" or "instruction" for what to do. A function can take input data (parameters) and return a result.

Example with TypeScript:

```
function greet(name: string) {
  console.log("Hello, " + name + "!");
}
greet("Ivan"); // Calling the function with the parameter "Ivan" - "Hello, Ivan!"
```

In this example, the `greet` function takes text (a name) as input and displays a message in the console.

What is a Class?
A class is a blueprint for creating objects. You can think of it as a "plan" or "blueprint" that describes the characteristics (properties) and actions (methods) an object will have. Every object created from the class follows this blueprint.

Example with TypeScript:

```
class Car {
  // Class properties
  make: string;
  model: string;
```

```
  // Constructor - a special function that creates a new object
  constructor(make: string, model: string) {
    this.make = make;
    this.model = model;
  }

  // Class method
  drive() {
    console.log(this.make + " " + this.model + " is driving.");
  }
}

// Creating an object from the Car class
let myCar = new Car("Toyota", "Corolla");
myCar.drive(); // "Toyota Corolla is driving."
```

Summary:
- Variable: A place to store data (like a "box").
- Variable Type: Determines what data you store (numbers, text, booleans, etc.).
- Function: A block of code that performs a task (like a "recipe" for an action).
- Class: A blueprint for creating objects that have properties and methods (like a "plan" for an object).

Asynchronous and Non Asynchronous data.

Lecture Level: Junior

What are Synchronous (Non-synchronous) Data?
Synchronous execution means tasks are carried out one after the other in a specific order. When a task starts, the program waits for it to finish before moving on to the next task. If a task takes time (like loading data from a server), the program will wait until the task is completed.

Example of synchronous code in JavaScript:

```
console.log("Starting...");

let result = slowFunction(); // The program waits for this function to complete
console.log(result); // This line runs only after slowFunction() returns a result

console.log("Finished!");
```

In this example, the program waits for each step to finish before moving forward. That can slow down execution, especially if you have time-consuming operations.

What is Asynchronous Data?
Asynchronous execution means tasks can run simultaneously, or rather, without waiting for previous tasks to finish. The program can continue executing other tasks while waiting for the asynchronous operation (e.g., a server request) to complete. Once the operation is done, the result is processed.

Asynchronous programming is beneficial when dealing with time-consuming operations, like fetching data from the internet, without blocking other parts of the program.

Example of asynchronous code in JavaScript:

```
console.log("Starting...");

setTimeout(() => {
  console.log("This will run after 2 seconds (asynchronously).");
}, 2000);

console.log("Finished!");
```

In this example, the program doesn't wait for 2 seconds before continuing. Instead, it moves to the following line while the asynchronous operation (setTimeout) runs in the background.

Difference Between Synchronous and Asynchronous Data:
- Synchronous: Tasks are executed one after another. The program waits for each task to complete before moving to the next.
- Asynchronous: Tasks can run concurrently or wait without blocking the rest of the program. The program can proceed while waiting for the result of an asynchronous operation.

Example with Asynchronous Data (Fetching Data from a Server):

```
console.log("Starting...");

async function getData() {
  try {
    // Waiting for the fetch request with await
    let response = await fetch("https://api.example.com/data");

    // Waiting for the response to be converted to JSON
    let data = await response.json();

    console.log("Received data:", data);
  } catch (error) {
    console.error("Error fetching data:", error);
  }
}

getData(); // Calling the asynchronous function

console.log("Finished!"); // This line runs while waiting for the asynchronous function
```

The program does not wait for the server request to finish before continuing. It handles the request's result when it’s ready.

Conclusion:
- Synchronous data: The program waits for each task to finish before moving to the next.
- Asynchronous data: The program does not wait for long tasks (like data loading) and continues executing other tasks while waiting for the result.

Asynchronous programming improves the performance of applications by preventing the program from being blocked during extended operations.

Domain Specific Language

Short story with a simple example

Lecture Level: Senior

What is a Domain-Specific Language (DSL)?
A Domain-Specific Language (DSL) is a programming language created to solve specific tasks within a particular domain. Unlike general-purpose programming languages like JavaScript, Python, or TypeScript, which can be used for a variety of tasks, DSL is designed for a specific purpose. A DSL simplifies the code, making it more readable and understandable to people working within the domain, even if they are not professional programmers.

Example of a Domain-Specific Language:
Imagine you want to write tests for a web application. Instead of using complex TypeScript syntax, you could create a DSL that makes these tests easy to read and write:

Example in TypeScript:

```
login("user", "password");
click("Submit");
expect(page).toContain("Welcome");
```

In this example, we have a DSL for testing a web application. Instead of writing complex functions using Playwright, this DSL abstracts those actions, making the test code easier to understand.

History of the DSL Term:
Paul Hudak was one of the first to introduce the term Domain-Specific Language (DSL) in the context of programming. A DSL is a specialized language designed to solve specific problems within a particular domain, unlike general-purpose languages like JavaScript or Python, which can be used for various tasks. Hudak explored how DSL could simplify programming, especially in areas like computer graphics and music, where specialized languages could make the code more understandable and less complex for developers.

Throughout his career, Paul Hudak developed and popularized the concept of embedded DSLs, where a specific language is integrated within a broader programming language. For example, he used Haskell to create DSLs that simplified tasks in specialized areas like computer animation and interactive graphics.

Why Use a DSL?
A DSL makes code easier to read, understand, and maintain. It is designed for people who may not be professional programmers but who work in a specific domain and need to solve domain-specific problems. For example, a DSL can be used in automated software testing to write tests that look more like natural language rather than complex code.

Domain-Specific Language in Automation QA with TypeScript and Playwright
When working with automated testing using Playwright and TypeScript, we often create our DSL to simplify test writing. DSL can make the code more readable and maintainable, even for people who aren't programming experts.

Example in the context of Playwright:
When writing tests with Playwright, you can create a DSL that abstracts away the complex details of interacting with the browser, such as clicking buttons, entering text, and verifying results. This helps make the tests easier to read and understand.

Example using Playwright and a DSL in TypeScript:

```
// DSL for user actions
async function loginUser(username: string, password: string) {
  await page.fill("username", username);
  await page.fill("password", password);
  await page.click("loginButton");
}

// DSL for verifying results
async function checkWelcomeMessage() {
  const message = await page.textContent("welcomeMessage");
  expect(message).toBe("Welcome back!");
}

// Using DSL for an easy-to-read test
await loginUser("testUser", "password123");
await checkWelcomeMessage();
```

Here, we've created DSL functions that abstract the logic for interacting with page elements and validations. Hiding the complexity of the Playwright interactions makes the test more readable.

Conclusion:
A Domain-Specific Language (DSL) is a powerful tool for automated testing with Playwright and TypeScript because it makes code more readable and understandable. DSL simplifies test creation by making the code more intuitive and less dependent on the complexity of the specific programming language. This helps QA teams write and maintain tests more efficiently, even if not all members are programming experts.

What exactly are we doing in common.ts?

Lecture Level: Senior

You can view the ./src/helpers/common.ts file of the boilerplate project.

Why do we need to group Playwright functions?

Lecture Level: Senior

Why Do We Group Playwright Functions and Use Them as Domain-Specific Language (DSL)?
When working with Playwright for automated testing, the main idea behind grouping functions and using them as a DSL is to make the code more readable, understandable, and easier to maintain. This practice abstracts away the complexity of Playwright and creates tests that are easier to manage and read. It is particularly useful when automating complex processes that require many steps and interactions with web page elements.

Benefits of Using DSL:
1. Readability and Maintainability: By grouping functions and presenting them in an easy-to-understand format, not only programmers but also QA engineers and other team members can understand the logic of the tests. This reduces the need for constant explanations or delving into the details of the Playwright API.
2. Code Reusability: Creating reusable functions saves time and effort, eliminating the need to rewrite the same actions repeatedly. For example, a function like `locateElement()` in your code can be reused in different contexts without rewriting the entire logic each time.
3. Clear Abstraction: DSL abstracts the details of Playwright operations and allows test code to be written at a higher level, focusing on business logic or specific test tasks. In your code, functions

like `clickElement()` or `sendKeys()` make the code easier to understand by hiding the details of interactions with DOM elements.
4. Easier Debugging: When key actions are encapsulated in standalone functions, such as in your Common class, if something goes wrong with a particular operation, you can easily find and fix the issue in one place without reviewing the entire project.

Example in TypeScript:

```
async sendKeys(
  locatorElement: string | Locator,
  text: string
) {
  const locator = await locateElement(locatorElement)
  await page.keyboard.press('Control+A')
  await page.keyboard.press('Meta+A')
  await locator.fill(text, { timeout: this.config.timeout })
  expect(await locator.inputValue()).toEqual(text)
  console.log('Successfully sent keys ' + text)
}
```

Instead of writing all these steps manually in each test, you simply call `sendKeys()`. This makes the code easier to read and maintain.

Notice that in the above `sendKeys()` function, you're also using another DSL-defined function—`locateElement()`.

```
async locateElement(
  element: Locator,
): Promise<Locator> {
  await element.focus({ timeout: this.config.timeout })
  await expect(element).toBeVisible({ timeout: this.config.timeout })
  await expect(element).not.toBeHidden({ timeout: this.config.timeout })
  await expect(element).toBeEnabled({ timeout: this.config.timeout })
  await expect(element).not toBeDisabled({ timeout: this.config.timeout })
  await expect(element).toHaveCount(1, { timeout: this.config.timeout })
  informLog('Successfully located element')
  return element
}
```

Why Use DSL in Playwright Automation?
1. Automated tests should be readable and understandable to all team members, not just developers.
2. Abstracting the complexity of Playwright allows for faster test creation and easier debugging.
3. Repetitive operations, such as locating and interacting with elements, can be encapsulated in DSL, making tests more elegant and efficient.

Conclusion:
Grouping Playwright functions as part of DSL simplifies automated testing by making tests more readable, maintainable, and understandable by the entire team. When tests are written using DSL, they become easier to read and manage, even for those who are not programming experts.

Page Object Model Design Pattern

What is Design Pattern, and why is it good to be used in programming?

Lecture Level: Mid

What is a Design Pattern?
A Design Pattern is a proven solution to a common problem in software development. It is not specific code but rather a template or strategy that you can use to organize and structure your code more effectively. Design patterns are helpful because they provide structured approaches to solving frequent programming problems, which makes code easier to understand and maintain.

Why do we use Design Patterns in programming?
1. Reusability of solutions: Instead of inventing a solution whenever you encounter a similar problem, you can use already proven patterns.
2. Improved readability: When you use well-known design patterns, the code becomes easier for other developers to understand because they recognize the pattern.
3. Enhanced maintainability: Patterns offer structure, making the code easier to maintain and extend in the future.

Types of Design Patterns
There are three main categories of design patterns:
1. Creational patterns: These patterns focus on how objects are created.
2. Structural patterns help organize classes and objects into more complex structures.
3. Behavioral patterns: These patterns deal with object interaction and responsibility.
By applying these patterns, developers can tackle recurring challenges in a more efficient and standardized way.

Why is the POM Design Pattern's most helpful design pattern in Automation Testing?

Lecture Level: Mid

Example with TypeScript and Playwright: Page Object Model (POM)
One of the most commonly used design patterns in Playwright test automation is the Page Object Model (POM). This model structures tests so that each web element or group of elements from a given web page is represented as a class. This class includes methods that encapsulate the actions that can be performed on these elements (such as clicking, entering text, etc.).

Why is POM useful?
- Separation of test logic from user interface logic: This makes the tests easier to maintain and less dependent on changes in the web page structure.
- Reusability: You can use the same logic for interacting with elements at different places in the tests.
- Improved readability: The test logic becomes easier to understand because you separate the interaction with UI elements into a separate class.

Example of Page Object Model (POM) with TypeScript and Playwright:

```
// The class representing the login page
class LoginPage {
  page: Page;

  constructor(page: Page) {
    this.page = page;
  }

  // Method for filling in the username field
  async enterUsername(username: string) {
    await this.page.fill('username', username);
```

```
  }

  // Method for filling in the password field
  async enterPassword(password: string) {
    await this.page.fill('password', password);
  }

  // Method for clicking the login button
  async clickLogin() {
    await this.page.click('loginButton');
  }
}

// Using POM in tests
async function loginTest(page: Page) {
  const loginPage = new LoginPage(page);

  // Use the methods of LoginPage for testing
  await loginPage.enterUsername('testUser');
  await loginPage.enterPassword('password123');
  await loginPage.clickLogin();
}
```

Conclusion:
Design Patterns help create more structured and efficient code that is easy to maintain and understand. In the context of TypeScript and Playwright, the Page Object Model (POM) is one of the most frequently used models, structuring interaction with web pages to make tests easier to manage.

How is it implemented in our testing framework?

Lecture Level: Mid

You can view the .\src\pages\examples\ExamplePage.ts and .\src\pages\templates\TemplatePage.ts files of the boilerplate project.

What is the idea of a base Page? Why are we using Base Page?

Lecture Level: Mid

What is a Base Class?
The base class (also known as a superclass) is a class that contains common functionalities and serves as a foundation for other classes. Classes that inherit from this base class are called subclasses. They automatically receive all the methods and properties of the base class but can add or modify their own. That makes the code more efficient and easier to maintain, as it helps avoid code duplication.

Why do we use base classes?
1. Code Reusability: Once the base class is created, you can inherit its methods and properties in various subclasses instead of rewriting them.
2. Easier Maintenance: If you need to change a common functionality, you can do it in the base class, and it will reflect in all subclasses.
3. Structured Code: Base classes provide a clear structure and organization in the code, especially in large projects.

Example of a suitable BasePage class in Playwright:
We can create a base class with Playwright that serves as a foundation for all "pages" in automated tests. It provides common functionalities such as access to the Playwright Page object and other frequently used objects.

Example of a base class with TypeScript:

```
export class BasePage {
    public page: Page;
    constructor(page: Page) {
        this.page = page;
    }
}
```

In this base class, we have:
- Page: This is the Playwright object that represents the web page we are working on within the tests.

How does inheritance work?
When you create a new class for a specific page, like LoginPage, it will inherit all the methods and properties from BasePage. That means there is no need to redefine the page object. It can be used directly because it is inherited.
The new class adds additional code to the particular test case, such as new methods for basic actions like navigation or clicking on an element.

Example of a subclass that inherits from BasePage with TypeScript:

```
class LoginPage extends BasePage {
    async login(username: string, password: string) {
        await this.page.fill('username', username);
        await this.page.fill('password', password);
        await this.page.click('loginButton');
    }
}
```

Here, LoginPage inherits all methods and properties from BasePage and adds a login method.

Conclusion:
Base classes are a fundamental concept in object-oriented programming, allowing for cleaner and more organised code. In your Playwright project, the BasePage base class facilitates the reuse of everyday actions, such as interacting with elements and navigation, making the tests more readable and easier to maintain.

You can view the .\src\pages\examples\ExamplePage.ts file of the boilerplate project.

What is an element?

Lecture Level: Junior

What is an Element?
An element is a fundamental part of a webpage. In the context of HTML and the DOM (Document Object Model), each element represents a specific webpage component— a paragraph, button, image, link, form, and more.

Example of an element (HTML):

```
<button>Click Me</button>
```

Here, `<button>` is the element that represents a button on the webpage.

What is an attribute?

Lecture Level: Junior

What is an Attribute?
An attribute provides additional information about an HTML element. It is used to define extra characteristics of the element, such as a unique identifier (ID), class, source of an image (src), link (href), and others.

Example of an element with attributes (HTML):

```
<button id="submitBtn" class="btn-primary">Click Me</button>
```

Here:
- `id="submitBtn"` is an attribute that assigns a unique identifier to the button.
- `class="btn-primary"` is an attribute that assigns a CSS class to style the button.

What is an attribute value?

Lecture Level: Junior

What is an Attribute Value?
The value of an attribute is the information it holds. For example, if the attribute is `id`, its value might be `submitBtn`, as seen in the example above. Attribute values are often used to identify or modify elements in the DOM via JavaScript or CSS.

Example with HTML:

```
<button id="submitBtn" class="btn-primary">Click Me</button>
```

- The value of the `id` attribute is `"submitBtn"`.
- The value of the `class` attribute is `"btn-primary"`.

What is a DOM tree?

Lecture Level: Junior

What is a DOM Tree?
The DOM (Document Object Model) is a hierarchical structure representing an HTML document as a tree of nodes. Each node in the DOM tree represents a part of the document—such as an element, attribute, or text. The DOM tree is how browsers "understand" and manage web pages, allowing them to display content on the screen.

How does the DOM work?
When a browser loads an HTML page, it creates a DOM tree where each HTML element is represented as a node. The DOM allows programmers to access and manipulate the webpage using JavaScript—adding, removing, or modifying elements.

Example of a DOM tree:
Given an HTML document structured as follows:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Header</h1>
    <p>This is a paragraph.</p>
```

```
  <button>Click me</button>
 </body>
</html>
```

The DOM tree for the above HTML code would be visualized like this:

```
└── body
    ├── h1 ("Header")
    ├── p ("This is a paragraph")
    └── button ("Click me")
```

What is a locator?

Lecture Level: Junior

What is a Locator?
A Locator is a method used to find and identify an element on a web page in order to interact with it in automated tests. In the context of Playwright and other automation tools like Selenium, locators help find elements so that actions like clicking, entering text, or verifying their state can be performed.

Different ways to locate an element. The best and worst approaches for locating an element (from ID to Class).

CSS Selector

XPath

Lecture Level: Junior, Mid

Different Ways to Locate an Element
There are several ways to locate an element on a web page, each with advantages and disadvantages. Here are the main methods:

1. ID
This is the most reliable way to locate an element with a unique ID. Each element on a page can have an `id` attribute, which should be unique to the page.
- Advantage: If the ID is unique and stable, this is the fastest and most accurate way to locate an element.
- Disadvantage: IDs may change dynamically (generated by JavaScript) in some cases, making this method unreliable.
Example in TypeScript:

```
 await page.locator('submitBtn').click();
```

Here, `submitBtn` is the ID of the button.

2. Class
You can use CSS classes to locate elements. Classes are usually used for styling elements but are not always unique, which may lead to locating multiple aspects simultaneously.
- Advantage: Easy to use when there is no unique ID.
- Disadvantage: A class is not always unique. Many elements can share the same class.
Example in TypeScript:

```
 await page.locator('.btn-primary').click();
```

Here, `.btn-primary` is the class of the button.

3. CSS Selector

CSS selectors provide a powerful and flexible way to locate elements. They can combine different attributes (IDs, classes, attributes) to find elements precisely.
- Advantage: A very flexible and powerful method to locate elements.
- Disadvantage: If the selector is too complex, it can become difficult to read and maintain.
Example in TypeScript:

```
await page.locator('div.container > ul > li.item').click();
```

Here, the selector chooses an element with class `item` that is a child of `ul` and inside a `div` with class `container`.

4. XPath
XPath is a query language used to locate elements in the structure of XML or HTML. XPath selectors can be very powerful, especially when an element has no unique identifier or class.
- Advantage: This can be used to find elements deeply nested in the DOM that don't have unique identifiers.
- Disadvantage: XPath selectors are often longer and more complex, making them harder to maintain. They can also be slower to execute compared to CSS selectors.
Example in TypeScript:

```
await page.locator('//button[text()="Click me"]').click();
```

Here, the XPath selector finds a button containing "Click me."

Best and Worst Approaches for Locating an Element:
- Best Approach:
  - ID: If a unique ID is available, this is the best approach as it is fast and reliable.
  - CSS Selector: A good choice, especially when using short and clear selectors.
- Weaker Approach:
  - Class: Be cautious when using a class that isn't unique, as it may locate multiple elements at once.
  - XPath: Complex XPath selectors are more challenging to maintain and are often slower than CSS selectors, but that is the most powerful approach to locating an element because it is a language specially created for selecting elements from XML documents. There are a lot of predefined functions and special keywords called Xpath Axes.

The case in which we can NOT locate an element by ID attribute (when the JS generates a different ID after reloading the page).

Lecture Level: Senior

When can we not use IDs?
In some cases, the IDs of elements may be dynamically generated by JavaScript. This means that with every page reload or change in state, the ID can change. In such cases, using an ID to locate an element will be unreliable because the code will not find the element on subsequent executions.

Example of a dynamic ID (HTML):

```
<button id="btn-12345">Click Me</button>
```

If upon reloading the page the ID becomes `btn-67890`, the locator by ID will no longer work. In such a case, you can use a CSS selector or XPath to find the element by other characteristics.

Stable locators, and why do we need a stable locator?

Lecture Level: Senior

What is a stable locator and why do we need it?

A stable locator is a way to identify an element on a web page that remains reliable even when the design or structure of the page changes. This means that this locator will continue to find the element correctly regardless of changes in the HTML code, JavaScript dynamically generated elements, or changes in the web page structure.

The need for a stable locator arises in automated testing, where unstable or changing elements can lead to test failures, even when the web page functions correctly. Stable locators make tests more reliable and easier to maintain, as there is no need to change them often with site updates.

What makes a locator stable?
1. Uniqueness: The locator should identify only one element on the page. If it selects multiple elements, the tests can become unclear and lead to errors.
2. Stability over time: The locator must resist changes in the web page's structure or dynamically generated elements. An example of an unstable locator uses dynamically generated IDs.
3. Easy maintenance: The stable locator should be easy to understand and maintain so that other team members can also use it.

Examples of stable locators:
1. ID Locator:
When an element has a unique and stable ID, this is the best way to locate it. IDs are fast to search and very accurate.
HTML:
```
<button id="submitBtn">Submit</button>
```
Locator with TypeScript and CSS Selector:
```
await page.locator('#submitBtn').click();
```
Locator with TypeScript and XPATH:
```
await page.locator(`//button[@id='submitBtn']`).click();
```
Here `submitBtn` is stable because:
- The ID is unique to this element.
- It does not change when the page is reloaded.

2. CSS Selector Locator:
If the element's ID is unavailable or not unique, CSS selectors can be a good way to find elements. For example, combining classes and nested selectors can be a stable solution.
HTML:
```
<div class="container">
  <button class="btn btn-primary">Submit</button>
</div>
```
Locator with TypeScript and CSS Selector:
```
await page.locator('.container .btn-primary').click();
```
Here the selector is stable because:
- It uses classes, which are generally less likely to change than dynamic IDs.
- It combines classes and nesting to identify the element precisely.
Always keep in mind when using the ‘class’ attribute as a reference point for our locator. The reason is that modern Front End JavaScript frameworks apply different styles to a given element precisely through the value of the ‘class’ attribute.

3. XPath Locator:
XPath is a powerful method for finding elements based on their location in the DOM tree. It can be useful when there is no unique ID or class.
HTML:

```
<div class="form">
  <button>Submit</button>
</div>
```

Locator with TypeScript and XPATH:

```
await page.locator('//div[@class="form"]//button[text()="Submit"]').click();
```

This XPath is stable because:
- It uses the button's text content (`text()="Submit"`) to precisely locate the element.
- It uses a relative path, making the locator more flexible when there are changes in the structure.

When might an ID not be reliable?
IDs can sometimes be dynamically generated by JavaScript, making them unreliable. For example:
HTML:

```
<button id="btn-12345">Submit</button>
```

If every time the page is reloaded the button's ID changes (e.g., to `btn-67890`), the ID locator will stop working. In this case, you can use alternative methods like CSS selectors or XPath.

Why do we need a stable locator?
- Reliability of tests: If you use unstable locators (like dynamic IDs), tests will break often with minor changes to the page.
- Less maintenance: Stable locators reduce the need to frequently change test code.
- Flexibility: Stable locators provide flexibility with minor changes in the web page structure, without requiring an update to every test.

Conclusion
Stable locators are essential for reliable and robust automated tests. The best approach is to use an ID when it is stable and unique. If an ID is not available or changes, you can rely on CSS selectors and XPath, but they must be well-designed to avoid breaking tests with changes in the page structure.

A real example of creating a locator for elements used in the example.

Lecture Level: Junior

You can view the .\src\pages\examples\ExamplePage.ts file of the boilerplate project.

Creating POM function.

Lecture Level: Mid

You can view the .\src\pages\examples\ExamplePage.ts file of the boilerplate project.

Showing a real example of a function.

Lecture Level: Junior

You can view the .\src\pages\examples\ExamplePage.ts file of the boilerplate project.

Different parts of the function (head, external properties, body, function type, returning something).

Lecture Level: Junior

You can view the .\src\pages\examples\ExamplePage.ts file of the boilerplate project.

Inside the POM function, we can show how we reuse the function from the DSL (common.ts) file.

Lecture Level: Senior

You can view the .\src\pages\examples\ExamplePage.ts file of the boilerplate project.

Lecture Level: Junior, Mid

What are Assertions?
Assertions are statements or checks in the code that verify that a certain condition is true. In the context of automated testing, assertions are used to confirm that a test has been executed correctly and that the program or web application operates according to expectations.

If an assertion fails, it means that the expected behavior of the application does not match the actual behavior, and the test is considered unsuccessful. Assertions are the primary way automated tests validate the functioning of the application.

What is the purpose of Assertions?
Assertions are used for:
1. Checking results: We ensure that the actual result of a given operation matches the expected result.
2. Detecting errors: If the results do not meet expectations, an assertion signals this with a failed test.
3. Enhancing reliability: Testing with assertions makes tests more reliable, providing clear indicators of whether the code is functioning correctly.

Two approaches to using Assertions in automated testing:
1. Assert at every small step (Step-by-step assertions):
This approach involves checking every single step in the test. With this method, we make an assert after every interaction with the application to ensure that each step is executed correctly.

Example:
- After filling in a text input field, we check if the text has been filled in correctly:
Example with TypeScript:

```
await page.fill('username', 'testUser')
await expect(page.locator('username')).toHaveValue('testUser') // Assertion for correct filling
```

- After marking a radio button, we make an assert to check if the button is marked correctly:
Example with TypeScript:

```
await page.click('radioOption')
await expect(page.locator('radioOption')).toBeChecked() // Assertion to check the selection
```

Advantages of this approach:
- Detailed verification: Each stage of the test is validated, which helps to detect errors immediately after they occur.
- Easy problem detection: If a step is not executed correctly, the assert immediately shows where the error occurred.

Disadvantages:
- Longer tests: Adding assertions after each step can make the tests longer and more complex.
- More maintenance: If the interface changes, many assertions may need to be updated.

2. Assert on the outcome (Final outcome assertions):
This approach involves one final check at the end of the test, which verifies that the ultimate goal has been achieved. Instead of checking each individual step, we check only whether the entire test was completed.
Example:

- After registering a new user, we can make an assertion that the registration was successful by checking the database, attempting to log in, or validating a successful registration message:
Example with TypeScript:

```
 await page.fill('username', 'newUser')
   await page.fill('password', 'newPassword')
   await page.click('registerButton')
   await expect(page.locator('.success-message')).toHaveText('Registration is successful') //
 Assertion for the final outcome
```

Advantages of this approach:
- Shorter tests: Fewer assertions make the tests shorter and easier to maintain.
- Focus on the goal: Instead of checking every step, we focus on the main goal of the test.

Disadvantages:
- Harder to find errors: If the test fails, it can be harder to determine exactly where the error occurred because we are checking only the outcome.

In automated testing, we can combine both approaches, thus making the tests more successful because they perform checks on every action executed, as well as checking whether the test has successfully concluded.

You can view the .\src\pages\examples\ExamplePage.ts file of the boilerplate project for more details.

Test Page

What is the idea of a test page?

Lecture Level: Junior, Mid

What is a Test Page (spec file)?
In test automation, a test page (called a more 'spec' file) is a class or structure within an automation testing framework responsible for interacting with a specific web page. The idea behind a Test Page is to encapsulate and organize the logic for interacting with the web page into a unified structure, making the tests easier to maintain and read.

Example structure of a test class in TypeScript:
Test classes typically contain methods to interact with page elements and test cases that verify their correct functioning.

```
 import { test, expect } from '@playwright/test';
 import { LoginPage } from './pages/LoginPage'; // Importing the Page Object Model class

 test.describe('Login Page Tests', () => {
   let loginPage: LoginPage;

   test.beforeEach(async ({ page }) => {
     loginPage = new LoginPage(page);
     await loginPage.navigateTo(); // Navigate to the login page
   });

   test('Should login successfully with valid credentials', async () => {
     await loginPage.enterUsername('testUser');
     await loginPage.enterPassword('testPassword');
```

```
    await loginPage.clickLoginButton();
    await expect(page).toHaveURL('/dashboard'); // Verify we are on the correct page
  });
});
```

In this example, the Test Page structure helps organize the interactions with the login page, while the test cases verify that these interactions result in the expected behavior (like a successful login).

You can view the .\src\tests\Example\Example.test.ts and .\src\tests\templates\TestTemplate.test.ts files of the boilerplate project to see the actual implementation.

Show an example of the structure of the test class.

The main block of the testing framework. The priority of executing different test blocks.

test.describe

test.beforeEach

test.afterEach

test

test.step

Other test blocks.

Lecture Level: Junior

You can view the .\src\tests\templates\TestTemplate.test.ts file of the boilerplate project to see the actual implementation.

Testing Data. Different approaches to testing data.

Lecture Level: Junior, Mid, Senior

Testing Data: Different Approaches to Testing Data
When writing automated tests, testing data refers to the information you use to run your tests. It could be usernames, emails, addresses, or any other type of data needed to simulate real-world scenarios in your application. There are two main approaches to testing data: static and dynamic.

Static testing Data.

Lecture Level: Junior

Static Testing Data
Static testing data means the data is fixed and doesn't change when you run the test. It's often used when you want to ensure consistency in your tests.

Hardcoded testing data.

Lecture Level: Junior

Hardcoded Testing Data
- What it is: This is the simplest approach where the data is directly written in the test script.
- Example: You might hardcode a username like `const username = 'testUser';`.
- Why it's useful: Hardcoded data is quick and easy to set up, and you know exactly what your data is every time you run the test.
- Drawbacks: Since the data doesn't change, tests may not cover a wide variety of real-world scenarios, and it can become repetitive.

Reading testing data from external resources like JSON, XML, excel files, or databases.

Lecture Level: Junior

Reading Testing Data from External Resources
- What it is: Instead of hardcoding data in your test, you can store it in external files (like JSON, XML, Excel, or even databases) and have your test read the data from there.
- Example: You can create a `data.json` file with the following content:
JSON:

```
{
 "username": "testUser",
 "email": "test@example.com"
}
```

Then, in your test, you read the data from this file.
- Why it's useful: This makes it easier to manage large amounts of data and allows you to separate the data from your test logic. It also simplifies updating data since you only need to modify the external file.
- Drawbacks: It can be slightly more complex to set up than hardcoding, especially if you're new to working with files or databases.

Dynamic testing data.

Lecture Level: Mid

Dynamic Testing Data
Dynamic testing data changes every time you run the test. This approach is useful when you want your tests to simulate different user actions or handle a variety of data inputs.

Using packages like FakerJS.

Lecture Level: Mid

Using Packages like FakerJS
- What it is: Instead of manually creating data, you use libraries like FakerJS to automatically generate random but realistic data, like names, emails, or addresses.
- Example with JavaScript: With FakerJS, you can generate a random name like this:

```
const fullName = faker.name.fullName();
```

- Why it's useful: This helps to simulate real-world scenarios more effectively, as your tests use different data each time, which can expose more potential issues in your application.
- Drawbacks: It can be harder to track issues if your tests fail, as the data changes with each run.

Generating different data every time by adding UUID or UNIX Time to testing data.

Lecture Level: Senior

Generating Different Data Every Time (UUID or UNIX Time)
- What it is: You can generate unique data each time by adding something like a UUID (Universally Unique Identifier) or UNIX Time (a timestamp) to your test data.
- Example with JavaScript: If you're testing a form and need a unique email address every time, you could generate an email like this:

```
const email = `user_${Date.now()}@example.com`; // Adds the current time to make it unique
```

- Why it's useful: This guarantees unique data for every test run, which prevents conflicts (like trying to create two accounts with the same email) and makes the test more robust.
- Drawbacks: Like using Faker, this can make it harder to debug tests since the data is different each time.

Randomly selecting data from drop-downs, radio buttons, checkboxes, etc.

Lecture Level: Senior

Randomly Selecting Data (Drop-downs, Radio Buttons, Checkboxes)
- What it is: For tests involving forms with multiple options (like drop-down menus, radio buttons, or checkboxes), you can write code to randomly select an option each time the test runs.
- Example with JavaScript:

```
const options = ['Option1', 'Option2', 'Option3'];
  const randomOption = options[Math.floor(Math.random() * options.length)];
```

- Why it's useful: This ensures your tests cover a wide range of options and simulate how different users might interact with your application.
- Drawbacks: The randomness can introduce unpredictability in your tests, making it harder to reproduce specific failures.

Calling the functions defined in the POM class.

Lecture Level: Senior

You can view the .\src\tests\Example\Example.test.ts

Debugging

What is debugging

Lecture Level: Junior

What is debugging?
Debugging is the process of finding and correcting errors (or bugs) in the code of a computer program. It involves investigating the causes of the program's unwanted behavior, analyzing the code, and fixing the issues to ensure the program runs correctly.

What does the debugging process involve?
1. Identifying the error: When the program does not behave as expected, you first need to discover the error. This could be an obvious error like a program crash or a hidden one where the program does not perform exactly as expected.

2. Isolating the error: Once the error is identified, you need to determine where in the code the problem lies. This is done by analyzing different parts of the code, using debugging tools (such as breakpoints and logging), and running the program step by step.

3. Finding the cause of the error: After isolating the error, you need to understand why it occurred and what caused it. This requires understanding the logic of the code and how its different parts interact.

4. Fixing the error: Once you understand the cause of the error, you need to modify the code so that the program behaves correctly.

5. Testing: After the fix, the program is tested again to ensure that the issue is resolved and that no new problems have arisen as a result of the change.

Why is debugging important?
Debugging is a crucial part of software development because it helps identify and correct issues before the software is used by end users. Bugs in the code can cause serious problems, such as incorrect program behavior, data loss, or security vulnerabilities.

What tools do we use for debugging?
- Breakpoints: These are points in the code where execution stops, allowing the programmer to examine the current state of the program—values of variables, memory state, etc.
- Logging: This involves recording information about the program's execution (e.g., variable values), which helps track what is happening.
- Interactive debuggers: Tools like the built-in debuggers in VSCode, Chrome DevTools, and other IDEs allow programmers to step through the code and examine the system's state.

Debugging in the context of JavaScript/TypeScript and Playwright
In JavaScript/TypeScript, debugging can be more challenging due to the dynamic nature of the language and code asynchronicity. When using Playwright (or another test framework), debugging is essential for identifying problems in automated tests, especially when dealing with synchronization issues or incorrect interaction with elements on web pages.

Example debugging tools in Playwright:
- `playwright pause`: You can pause the test and inspect the state of the page during test execution.
- `console.log()`: Logging data in tests helps you understand what data is being passed or returned during the test.

Why is debugging in JS so hard (because compilation happens before the execution)?

Lecture Level: Mid

Why is debugging in JavaScript more difficult?
Debugging in JavaScript can be more challenging than in other languages due to several key reasons:

1. Asynchronous Nature
One of the main reasons why debugging in JavaScript is more complicated is the asynchronous nature of the language. In JavaScript, we often use functions like `setTimeout()`, `fetch()`, and Promises, which are not executed immediately but at a later stage in the program. This means that the program continues running while waiting for the result of these operations and debugging in such a context can be confusing.

Example in JavaScript:
```
setTimeout(() => {
  console.log('This function will be executed later');
}, 2000);

console.log('This function executes immediately');
```

When debugging such code, it can be hard to track exactly where and when the errors occur because the operations are not executed sequentially.

2. Dynamic Nature
JavaScript is a dynamic language, meaning variables can change their types during execution. This can lead to unexpected errors that do not appear at compile time. Unlike other languages like TypeScript or Java, where types are strictly defined, in JavaScript, the same variable can hold a number, a string, or an object at different times.

Example in JavaScript:

```
let data = "5";
data = data + 5; // This will return "55" instead of 10, which can lead to unexpected results
```

These errors can be difficult to detect because the language does not warn about type mismatches.

3. No Compilation in JS
In JavaScript, no compilation step warns about errors before running the code. This means that errors are often detected during runtime, which can make them harder to identify and fix. In many other languages, the compiler will warn you about errors in the code before it is executed.

This makes debugging in JavaScript more difficult because you have to rely on tools like console logs or debuggers to track what exactly is happening in the code.

4. Hoisting
In JavaScript, hoisting means that variable and function declarations are "lifted" to the top of their scope before the code is executed. This can confuse because variables seem to "exist" before they are declared, but they may not have a value.

Example in JavaScript:

```
console.log(a); // Undefined
var a = 5;
```

This behavior can lead to unexpected errors that are difficult to trace.

5. Modules and Global Scope
Historically, JavaScript used a global scope (especially before the introduction of modules), meaning that all variables could be accessed anywhere in the code. This can create problems during debugging, as variables can be accidentally changed or reused in different parts of the program.

What is our solution for debugging?

Lecture Level: Mid

Another way to debug in Playwright is by using the Playwright Test for VSCode extension. You can set a breakpoint and start the debugger using the extension.

Executing the tests from the terminal.

Lecture Level: Junior

Test Execution Methods
Tests can be executed in various ways, but the most appropriate method is through terminal commands. We've already mentioned the benefit of the scripts section in the package.json file. Here's an example command:

JSON:

```
"scripts": {
  "test:register": "playwright test RegisterNewUser.test.ts"
}
```

As you can see, the `test:register` command essentially triggers another, longer command. In practice, it doesn't matter which command you execute — both will lead to the same result.

Why should we run tests via terminal commands?

If you are currently developing or debugging a test, running it through a debugging tool is best. However, if you're not developing or troubleshooting a specific test, it's better to run the test(s) using a terminal command. This is because when the test enters the Pipeline, it will be executed through such (or similar) terminal commands.

Creating new script records inside the package.json.

In Playwright, we often use `package.json` to add "scripts" that allow us to easily run different tests or commands with a single command in the terminal.

What is a script in package.json?
The `package.json` file is a configuration file for Node.js projects that stores information about the project, its dependencies, and the scripts we can run. A script is simply a shortcut to execute a specific command. For example, if we want to quickly run Playwright tests, we can create a script for that.

How to create a new Playwright script in package.json?
1. Open the `package.json` file:
   You can find `package.json` in the main directory of your project.

2. Add a new script in the "scripts" section:
   In `package.json`, there's a `"scripts"` section where we can add new commands. Here's how to add a few useful Playwright scripts:
JSON
```
  "scripts": {
      "test": "npx playwright test",
      "test:ui": "npx playwright test --ui",
      "test:headed": "npx playwright test --headed",
      "test:login": "npx playwright test --grep '@login'"
  }
```
- "test": This is the main Playwright test script. The command `npx playwright test` will run all tests.
- "test:ui": This script launches the Playwright Test Runner UI, which allows you to run tests through a graphical interface. It's useful for visually managing tests.
- "test:headed": If you want to run tests in a mode where the browser opens, use this script. `--headed` shows the browser during the tests.
- "test:login": This will run only the tests tagged with `@login`. This is convenient when you want to run only a subset of tests. You can tag your tests with `@login` in their descriptions so this script can detect them.

3. Running the scripts:
To execute a script, simply type `npm run` followed by the script name. For example:
Bash
```
  npm run test       # Runs all tests
  npm run test:ui    # Launches Playwright Test Runner UI
  npm run test:headed # Runs tests with the browser open
  npm run test:login  # Runs tests tagged with @login
```

These scripts make working with Playwright easier and more convenient, especially if you frequently use specific configurations. You can also add other scripts as needed, such as those for generating reports or running different test suites.

Automatically generated Reports after executing the tests.

Automatically Generated Reports after Executing the Tests in Playwright
Playwright provides the option to see a report of all tests after execution. Automatically generated reports gather information on how many tests passed and which failed and detailed information about each error.

By default, Playwright uses an HTML report for visualization, where we can easily review:
- An overview of the results: the number of successful and failed tests and the reasons for any failures.
- Details for each test: such as execution time and specific error information.
- Screenshots and videos: if we set up tests to capture videos or take screenshots of failure, we can view them directly in the report.

These reports are automatically created after the tests finish running and are often available in an HTML format, which is convenient for sharing with the team.

Grouping automation tests in Suites

Grouping Automation Tests in Suites in Playwright
In Playwright, we can organize tests into "suites" to group them according to different functionalities or goals. For example:
- Login Suite: includes tests for login functionality.
- Cart Suite: contains tests for the shopping cart.
- Checkout Suite: covers tests for the checkout process.

Creating suites allows us to run only the tests we need. In Playwright, this is done easily by organizing tests within code files or tagging tests by purpose.

Execute different suites.

Execute Different Suites in Playwright
Once we've organized tests into suites, Playwright lets us run specific suites or even individual tests as needed. This is particularly helpful when we want to test only a certain part of the application.

Examples of execution:
- Running the Login Suite: we can execute only the login tests if we make changes to this functionality.
- Running all test suites: when we want a full test of the application.
- Running specific suites: if we want to test only the cart and checkout.

This management is configured through Playwright's settings and terminal commands. For example, we can use `npx playwright test --project=<suite-name>` to run a specific suite. This flexibility helps save time and resources by focusing only on what is important at the moment.

API testing using Playwright.

Before we begin, it is important to note that API testing must be in a separate course. There is so much that can be learned on the subject that this course cannot possibly cover. However, we will get to know the API testing that Playwright can provide.

API Testing using Playwright

API (Application Programming Interface) testing is a type of testing that focuses on verifying the functionality, reliability, performance, and security of APIs. In API testing, instead of testing how the user interacts with the front end, you directly test the communication between different parts of the system (back end). API testing ensures that the API works correctly and returns the expected data when it is called.

a) What is API testing?
- API testing means sending a request (for example, asking for user data) to an API and checking if the response (for example, the user's name or ID) is correct.
- You test how data flows between different parts of your application or how your system communicates with external systems through APIs.

b) Why use Playwright for API testing?
Playwright is usually known for web automation, but it also provides tools to test APIs. Using Playwright for API testing allows you to test both the back-end APIs and the front-end (UI) interactions within the same tool, making it easier to manage tests.

The Playwright allows you to:
- Send API requests (like GET, POST, PUT, DELETE) to test if your back-end services work correctly.
- Verify if the response from an API matches the expected data (for example, checking if the correct user details are returned).

Example of a simple API test in Playwright using JavaScript:
Here's a basic example of testing an API with Playwright:

```
import { test, expect } from '@playwright/test';

test('API Test - Get user data', async ({ request }) => {
  // Send a GET request to the API endpoint
  const response = await request.get('https://api.example.com/users/1');

  // Expect the response status to be 200 (success)
  expect(response.status()).toBe(200);

  // Extract JSON data from the response
  const userData = await response.json();

  // Verify the data (for example, checking if the username is correct)
  expect(userData.username).toBe('john_doe');
});
```

In this example:
- We're sending a GET request to an API endpoint that returns user data.
- We check if the response status is 200 (which means success).
- Then, we verify if the returned data (in this case, the username) is correct.

Combining API tests and Web Automation tests.

Combining API Tests and Web Automation Tests:
One of the powerful features of Playwright is that it allows you to combine both API tests and Web Automation tests in a single flow. This is helpful when you want to test how your front end (the web page) works with your back end (the APIs) together.
a) Why combine API and Web Automation tests?

In real-world applications, the front end and back end work together. For example, when a user logs in on a website, the login form on the web page sends the username and password to an API, and the API checks if they are correct. By combining API tests with web tests, you can:
- Ensure that the API is working correctly.
- Ensure that the front-end interaction with the API is correct.
- Test a full-user scenario, such as filling out a form on the website and verifying that the data is correctly saved via an API.

b) Example of combining API and Web Automation testing:
Let's say you want to test the login process on a website and ensure that it works correctly both from the front-end and back-end perspectives.

Here's an example with JavaScript:

```
import { test, expect } from '@playwright/test';

test('Login Flow - API and Web Test', async ({ page, request }) => {
  // Step 1: API Test - Create a new user via API
  const userResponse = await request.post('https://api.example.com/users', {
    data: {
      username: 'test_user',
      password: 'password123'
    }
  });
  expect(userResponse.status()).toBe(201); // Ensure the user was created successfully

  // Step 2: Web Automation Test - Login as the new user
  await page.goto('https://example.com/login');
  await page.fill('input[name="username"]', 'test_user');
  await page.fill('input[name="password"]', 'password123');
  await page.click('button[type="submit"]');

  // Step 3: Verify the user is logged in by checking if the dashboard is shown
  await expect(page).toHaveURL('https://example.com/dashboard');
});
```

In this example:
1. API Test: First, we create a new user by sending a POST request to the API.
2. Web Automation Test: Then, we simulate the user logging in via the website's login form.
3. We verify that the user was successfully logged in by checking if the browser is redirected to the dashboard.

This combination ensures that both the API for creating a user and the web interface for logging in work as expected.

Summary:
- API Testing allows you to test the back-end services by sending requests and verifying the responses.
- Playwright lets you combine API testing and web automation to create complete test scenarios that check both the front end (the web page) and the back end (the APIs) together.
- Combining these tests helps you catch bugs that could occur when the front end and back end communicate.

By learning how to use Playwright for both API and web tests, you can create more comprehensive tests for your applications!

Updating the packages in the NodeJS project.

Lecture Level: Mid

Updating packages in a Node.js project, including a Playwright project, is a common task to ensure you're using the latest features and fixes. Here's how you can do it, explained step-by-step for beginners.

We already explained what is package.json file and what its purpose of it. Updating the packages is related to the package.json file. So make sure you understand what that package does.

1. Checking for outdated packages
To see if there are updates available for the packages in your project, run this command in the terminal:
Bash Command:

```
npm outdated
```

This will show a list of packages in your project that are outdated, along with the current and latest available versions.

2. Updating all packages to the latest versions
If you want to update all the packages in your project to their latest versions, run:
Bash Command:

```
npm update
```

This will update the packages to the latest minor or patch versions that are compatible with the version specified in your `package.json` file.

3. Updating packages to the latest major versions
To update packages to their latest major versions (which might include breaking changes), you can run:
Bash Command:

```
npm install package-name@latest
```

For example, to update Playwright to the latest version:
Bash Command:

```
npm install playwright@latest
```

You can also update all packages (both dependencies and devDependencies) to their latest versions by running:
Bash Command:

```
npx npm-check-updates -u
```

Then install the updated packages:
Bash Command:

```
npm install
```

4. Updating the Playwright itself
For Playwright-specific projects, you can update Playwright using the following command:
Bash Command:

```
npm install playwright@latest
```

After updating Playwright, you may also want to update the browsers that Playwright uses for testing. You can do this by running:
Bash Command:

```
npx playwright install
```

This command ensures that the latest browser versions compatible with the updated Playwright version are installed.

5. Testing after updates

After updating packages, especially major updates, it's important to test your project to ensure that everything still works correctly. Run your tests to verify that the updated versions didn't introduce any breaking changes.

For example, in Playwright, you can run:

Bash Command:

```
npx playwright test
```

Summary:

- `npm outdated`: Check which packages are outdated.
- `npm update`: Update to the latest compatible versions.
- `npm install package-name@latest`: Update specific packages to the latest major version.
- `npx playwright install`: Update Playwright browsers after updating Playwright itself.

By following these steps, you can keep your Node.js and Playwright project up to date and make sure your project benefits from the latest features and bug fixes!